

Lab Programs

15/7/26

1-6

Set 1: Monthly Sales Data

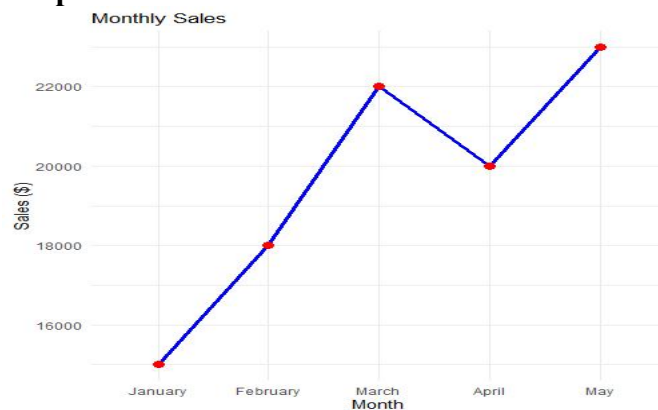
Month	Sales (in \$)
January	15000
February	18000
March	22000
April	20000
May	23000

1.A: Create a line chart to visualize the monthly sales. Label the axes and title the chart appropriately.

Code:

```
library(ggplot2)
sales_data <- data.frame(
  Month = factor(c("January", "February", "March", "April", "May"),
    levels = c("January", "February", "March", "April", "May")),
  Sales = c(15000, 18000, 22000, 20000, 23000)
)
# Line Chart
ggplot(sales_data, aes(x = Month, y = Sales, group = 1)) +
  geom_line(color = "blue", linewidth = 1.2) +
  geom_point(color = "red", size = 3) +
  labs(
    title = "Monthly Sales",
    x = "Month",
    y = "Sales ($)"
  ) +
  theme_minimal()
```

Output:

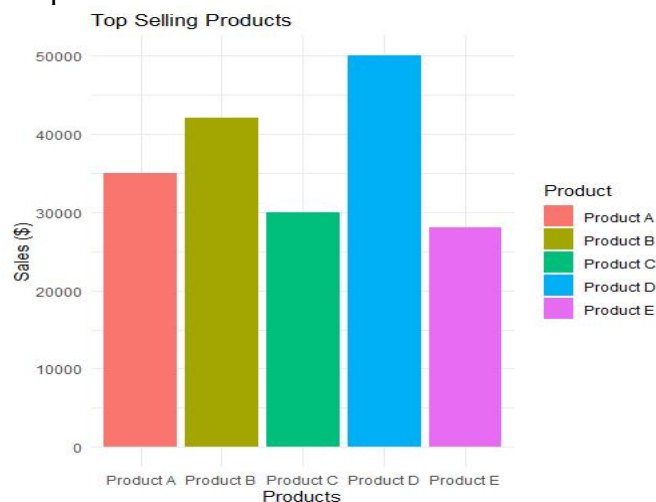


1.B:Generate a bar chart to display the top-selling products for the year. Label the chart elements.

Code:

```
library(ggplot2)
product_data <- data.frame(
  Product = c("Product A", "Product B", "Product C", "Product D", "Product E"),
  Sales = c(35000, 42000, 30000, 50000, 28000)
)
ggplot(product_data, aes(x = Product, y = Sales, fill = Product)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Top Selling Products",
    x = "Products",
    y = "Sales ($)"
  ) +
  theme_minimal()
```

Output:



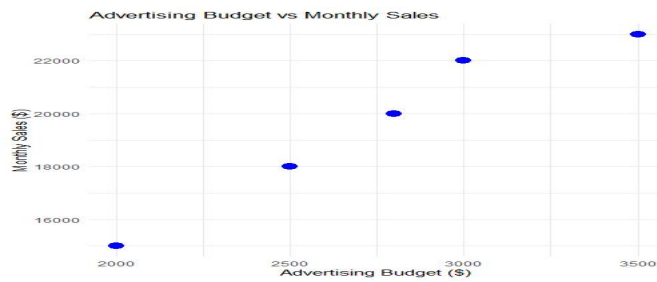
1.C:Develop a scatter plot to explore the relationship between advertising budget and monthly sales.

Explain the insights drawn from the scatter plot.

Code:

```
library(ggplot2)
scatter_data <- data.frame(
  Advertising = c(2000, 2500, 3000, 2800, 3500),
  Sales = c(15000, 18000, 22000, 20000, 23000)
)
ggplot(scatter_data, aes(x = Advertising, y = Sales)) +
  geom_point(color = "blue", size = 4) +
  labs(
    title = "Advertising Budget vs Monthly Sales",
    x = "Advertising Budget ($)",
    y = "Monthly Sales ($)"
  ) +
  theme_minimal()
```

Output:



1.D: Build an interactive dashboard combining the line chart and bar chart to allow users to explore sales data interactively.

Code:

```
install.packages("shiny")
library(shiny)
library(ggplot2)
sales <- data.frame(
  Month = factor(c("January", "February", "March", "April", "May"),
    levels = c("January", "February", "March", "April", "May")),
  Sales = c(15000, 18000, 22000, 20000, 23000)
)
products <- data.frame(
  Product = c("Laptop", "Mobile", "Tablet", "Headphones", "Smart Watch"),
  Sales = c(52000, 48000, 35000, 28000, 25000)
)
ui <- fluidPage(
  titlePanel("Monthly Sales Dashboard"),
  sidebarLayout(
    sidebarPanel(
      helpText("Interactive Sales Dashboard")
    ),
    mainPanel(
      plotOutput("lineChart"),
      br(),
      plotOutput("barChart")))
server <- function(input, output) {
  output$lineChart <- renderPlot({
    ggplot(sales,
      aes(x = Month,
        y = Sales,
        group = 1)) +
    geom_line(color = "blue", linewidth = 1.2) +
    geom_point(color = "red", size = 3) +
    geom_text(aes(label = Sales),
      vjust = -1) +
    labs(
      title = "Monthly Sales",
      x = "Month",
      y = "Sales ($)"
    ) +
    theme_minimal()})
  output$barChart <- renderPlot({
```

```

ggplot(products,
  aes(x = Product,
      y = Sales,
      fill = Product)) +
  geom_bar(stat = "identity") +
  geom_text(aes(label = Sales),
    vjust = -0.5) +
  labs(
    title = "Top Selling Products",
    x = "Products",
    y = "Sales ($)"
  ) +
  theme_minimal()
})
}
shinyApp(ui = ui, server = server)

```

Output:



2.Dataset: Customer Satisfaction

Customer ID	Age	Satisfaction Score
1	25	4
2	30	5
3	35	3
4	28	4
5	40	5

2.A:Create a histogram to represent the distribution of customer ages. Label the axes and title the chart.

Code:

```

library(ggplot2)
customer_data <- data.frame(
  CustomerID = 1:5,
  Age = c(25, 30, 35, 28, 40),

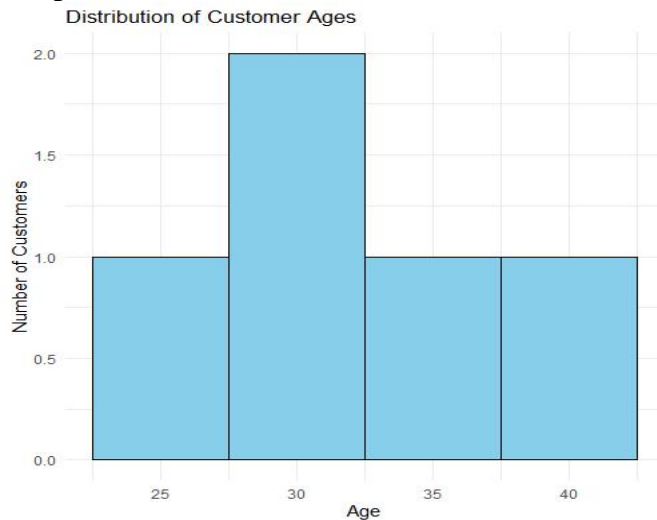
```

```

Satisfaction = c(4, 5, 3, 4, 5)
)
ggplot(customer_data, aes(x = Age)) +
  geom_histogram(binwidth = 5, fill = "skyblue", color = "black") +
  labs(
    title = "Distribution of Customer Ages",
    x = "Age",
    y = "Number of Customers"
  ) +
  theme_minimal()

```

Output:



2:B:Generate a pie chart to display the overall distribution of customer satisfaction scores. Include labels.

Code:

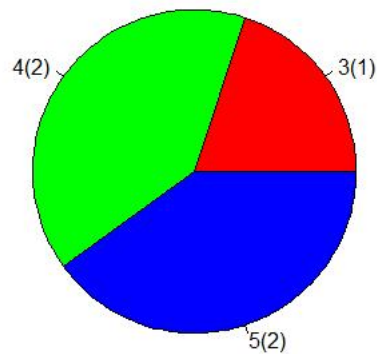
```

customer_data <- data.frame(
  Satisfaction = c(4, 5, 3, 4, 5)
)
score_count <- table(customer_data$Satisfaction)
pie(
  score_count,
  labels = paste(names(score_count), "(", score_count, ")", sep = ""),
  main = "Customer Satisfaction Scores",
  col = rainbow(length(score_count))
)

```

Output:

Customer Satisfaction Scores

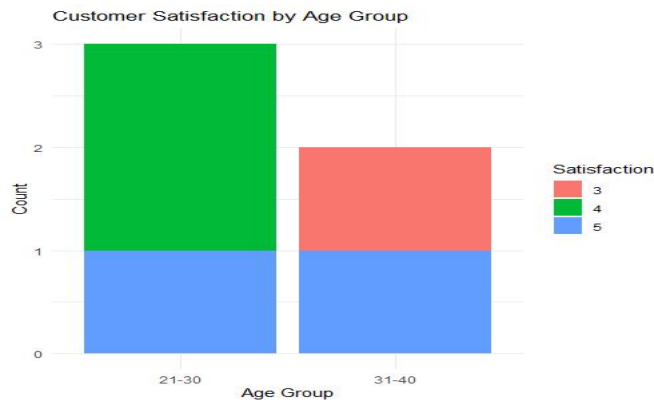


2:C: Build a stacked bar chart to visualize the distribution of customer satisfaction scores by age group.

Code:

```
library(ggplot2)
customer_data <- data.frame(
  Age = c(25, 30, 35, 28, 40),
  Satisfaction = c(4, 5, 3, 4, 5)
)
customer_data$AgeGroup <- cut(
  customer_data$Age,
  breaks = c(20, 30, 40),
  labels = c("21-30", "31-40"),
  include.lowest = TRUE
)
ggplot(customer_data,
  aes(x = AgeGroup,
    fill = factor(Satisfaction))) +
  geom_bar() +
  labs(
    title = "Customer Satisfaction by Age Group",
    x = "Age Group",
    y = "Count",
    fill = "Satisfaction"
  ) +
  theme_minimal()
```

Output:



2:D:Develop a word cloud from open-ended customer feedback to identify prevalent customer sentiments.

Code:

```
library(tm)
library(wordcloud)
feedback <- c(
  "Excellent service and friendly staff",
  "Very satisfied with the product",
  "Average experience",
  "Good quality and quick delivery",
  "Excellent support and service"
)
corpus <- Corpus(VectorSource(feedback))
corpus <- tm_map(corpus, content_transformer(tolower))
corpus <- tm_map(corpus, removePunctuation)
corpus <- tm_map(corpus, removeNumbers)
corpus <- tm_map(corpus, removeWords, stopwords("english"))
corpus <- tm_map(corpus, stripWhitespace)
tdm <- TermDocumentMatrix(corpus)
m <- as.matrix(tdm)
words <- rownames(m)
freq <- rowSums(m)
wordcloud(
  words = words,
  freq = freq,
  min.freq = 1,
  colors = rainbow(10)
)
```

Output:



Set 3: Employee Performance Evaluation

Dataset: Employee Performance

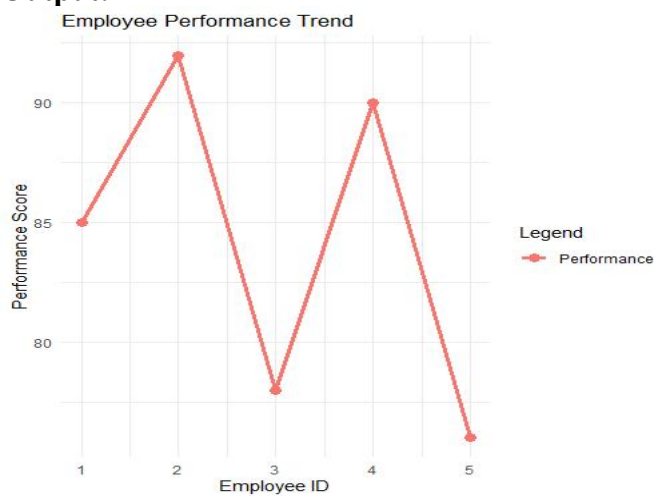
Employee ID	Department	Years of Service	Performance Score
1	Sales	5	85
2	HR	3	92
3	Marketing	7	78
4	Sales	4	90
5	HR	2	76

3.A: Create a line chart to visualize the performance trend of employees over time. Include a legend and labels.

Code:

```
library(ggplot2)
employee_data <- data.frame(
  EmployeeID = c(1,2,3,4,5),
  Department = c("Sales","HR","Marketing","Sales","HR"),
  YearsService = c(5,3,7,4,2),
  Performance = c(85,92,78,90,76)
)
ggplot(employee_data, aes(x = EmployeeID, y = Performance, group = 1, color =
"Performance")) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 3) +
  labs(
    title = "Employee Performance Trend",
    x = "Employee ID",
    y = "Performance Score",
    color = "Legend"
  ) +
  theme_minimal()
```

Output:

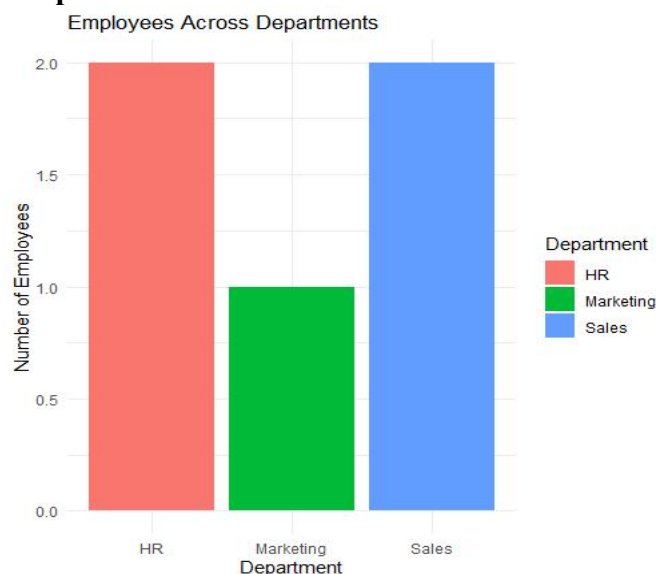


3.B:Generate a bar chart showing the distribution of employees across different departments. Label the chart elements.

Code:

```
library(ggplot2)
employee_data <- data.frame(
  EmployeeID = c(1,2,3,4,5),
  Department = c("Sales","HR","Marketing","Sales","HR"),
  YearsService = c(5,3,7,4,2),
  Performance = c(85,92,78,90,76)
)
ggplot(employee_data, aes(x = Department, fill = Department)) +
  geom_bar() +
  labs(
    title = "Employees Across Departments",
    x = "Department",
    y = "Number of Employees"
  ) +
  theme_minimal()
```

Output:



3.C:Build a scatter plot to analyze the correlation between years of service and performance scores. Explain any insights.

Code:

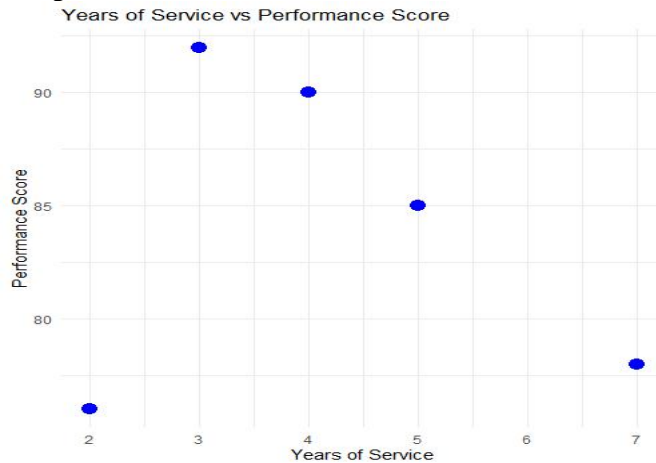
```
library(ggplot2)
employee_data <- data.frame(
  EmployeeID = c(1,2,3,4,5),
  Department = c("Sales","HR","Marketing","Sales","HR"),
  YearsService = c(5,3,7,4,2),
  Performance = c(85,92,78,90,76)
)
ggplot(employee_data,
  aes(x = YearsService, y = Performance)) +
  geom_point(color = "blue", size = 4) +
  labs(
    title = "Years of Service vs Performance Score",
```

```

x = "Years of Service",
y = "Performance Score"
) +
theme_minimal()

```

Output:



3.D: Develop a Tableau dashboard with the line chart and bar chart for interactive exploration.

Code:

```

library(shiny)
library(plotly)
employee_data <- data.frame(
  EmployeeID = c(1,2,3,4,5),
  Department = c("Sales", "HR", "Marketing", "Sales", "HR"),
  YearsService = c(5,3,7,4,2),
  Performance = c(85,92,78,90,76)
)
ui <- fluidPage(
  titlePanel("Employee Performance Dashboard"),
  fluidRow(
    column(6,
      plotlyOutput("linePlot")
    ),
    column(6,
      plotlyOutput("barPlot")
    )
  )
)
server <- function(input, output){
  output$linePlot <- renderPlotly({
    plot_ly(employee_data,
      x = ~EmployeeID,
      y = ~Performance,
      type = "scatter",
      mode = "lines+markers") %>%
    layout(
      title = "Employee Performance Trend",
      xaxis = list(title = "Employee ID"),
      yaxis = list(title = "Performance Score")
    )
  })
}

```

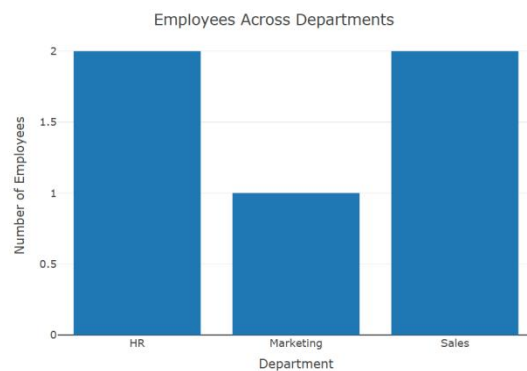
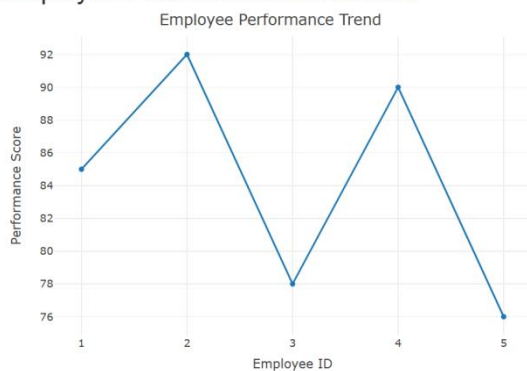
```

})
output$barPlot <- renderPlotly({
  dept_count <- as.data.frame(table(employee_data$Department))
  colnames(dept_count) <- c("Department", "Employees")
  plot_ly(dept_count,
    x = ~Department,
    y = ~Employees,
    type = "bar") %>%
  layout(
    title = "Employees Across Departments",
    xaxis = list(title = "Department"),
    yaxis = list(title = "Number of Employees")
  )
})
}
shinyApp(ui = ui, server = server)

```

Output:

Employee Performance Dashboard



Set 4: Product Inventory Management

Dataset: Product Inventory

Product ID	Product Name	Quantity Available
1	Product A	250
2	Product B	175
3	Product C	300
4	Product D	200
5	Product E	220

4.A: Create a bar chart to visualize the quantity of each product in the inventory.

Label the axes and title the chart.

Code:

```

library(ggplot2)
inventory_data <- data.frame(
  ProductID = c(1,2,3,4,5),
  ProductName = c("Product A","Product B","Product C","Product D","Product E"),
  Quantity = c(250,175,300,200,220)
)

```

```
ggplot(inventory_data,
  aes(x = ProductName, y = Quantity, fill = ProductName)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Product Inventory Quantity",
    x = "Product Name",
    y = "Quantity Available"
  ) +
  theme_minimal()
```

Output:

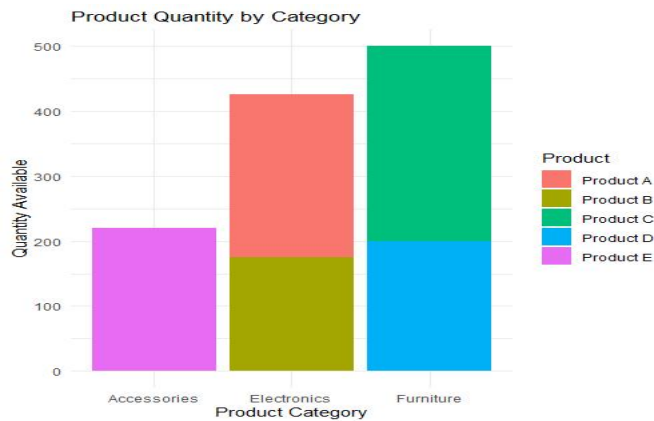


4.B:Generate a stacked bar chart to show the quantity of each product within different product categories.

Code:

```
library(ggplot2)
inventory_data <- data.frame(
  ProductName = c("Product A","Product B","Product C","Product D","Product E"),
  Quantity = c(250,175,300,200,220),
  Category = c("Electronics","Electronics","Furniture","Furniture","Accessories")
)
ggplot(inventory_data,
  aes(x = Category, y = Quantity, fill = ProductName)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Product Quantity by Category",
    x = "Product Category",
    y = "Quantity Available",
    fill = "Product"
  ) +
  theme_minimal()
```

Output:



4.C: Build a scatter plot to explore the relationship between product price and quantity available. Explain the findings.

Code:

```
library(ggplot2)
inventory_data <- data.frame(
  ProductName = c("Product A", "Product B", "Product C", "Product D", "Product E"),
  Price = c(500, 350, 700, 450, 600),
  Quantity = c(250, 175, 300, 200, 220)
)
ggplot(inventory_data,
  aes(x = Price, y = Quantity)) +
  geom_point(color = "blue", size = 4) +
  labs(
    title = "Product Price vs Quantity Available",
    x = "Product Price",
    y = "Quantity Available"
  ) +
  theme_minimal()
```

Output:



4.D: Develop a Tableau dashboard with the bar chart and stacked bar chart to allow users to interact with the data.

Code:

```
library(shiny)
library(plotly)
inventory_data <- data.frame(
  ProductName = c("Product A", "Product B", "Product C", "Product D", "Product E"),
```

```

Quantity = c(250,175,300,200,220),
Category = c("Electronics","Electronics","Furniture","Furniture","Accessories")
)
ui <- fluidPage(
  titlePanel("Product Inventory Dashboard"),
  fluidRow(
    column(6,
      plotlyOutput("barPlot")
    ),
    column(6,
      plotlyOutput("stackedPlot"))))
server <- function(input, output){
  output$barPlot <- renderPlotly({
    plot_ly(
      inventory_data,
      x = ~ProductName,
      y = ~Quantity,
      type = "bar"
    ) %>%
    layout(
      title = "Product Inventory Quantity",
      xaxis = list(title = "Product"),
      yaxis = list(title = "Quantity")
    )
  })
  output$stackedPlot <- renderPlotly({
    plot_ly(
      inventory_data,
      x = ~Category,
      y = ~Quantity,
      color = ~ProductName,
      type = "bar"
    ) %>%
    layout(
      barmode = "stack",
      title = "Quantity by Product Category",
      xaxis = list(title = "Category"),
      yaxis = list(title = "Quantity")
    )
  })
}
shinyApp(ui = ui, server = server)

```

Output:

Product Inventory Dashboard



Set 5: Website Analytics

Dataset: Website Traffic

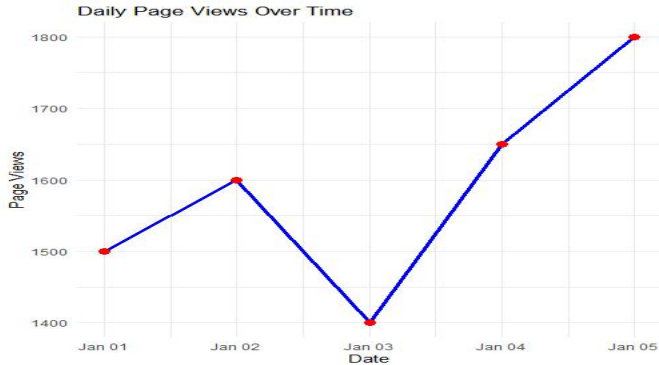
Date	Page Views	Click-through Rate
2023-01-01	1500	2.3%
2023-01-02	1600	2.7%
2023-01-03	1400	2.0%
2023-01-04	1650	2.4%
2023-01-05	1800	2.6%

5.A: Create a line chart to visualize the trend in daily page views over time. Label the axes and title the chart.

Code:

```
library(ggplot2)
website_data <- data.frame(
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03", "2023-01-04", "2023-01-05")),
  PageViews = c(1500, 1600, 1400, 1650, 1800),
  CTR = c(2.3, 2.7, 2.0, 2.4, 2.6)
)
ggplot(website_data, aes(x = Date, y = PageViews, group = 1)) +
  geom_line(color = "blue", linewidth = 1.2) +
  geom_point(color = "red", size = 3) +
  labs(
    title = "Daily Page Views Over Time",
    x = "Date",
    y = "Page Views"
  ) +
  theme_minimal()
```

Output:

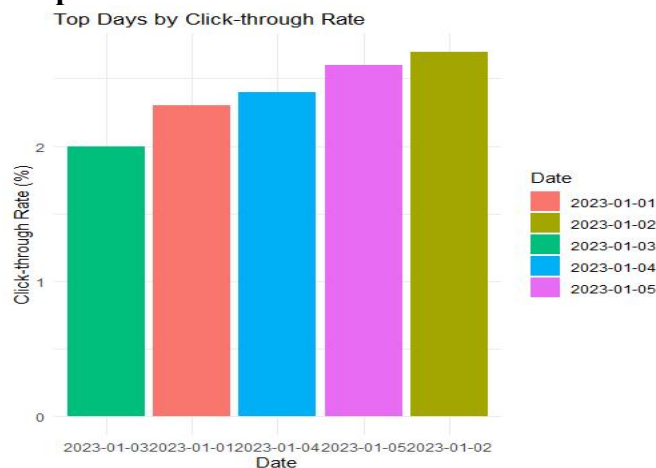


5.B:Generate a bar chart showing the top N days with the highest click-through rates. Label the chart elements.

Code:

```
library(ggplot2)
website_data <- data.frame(
  Date = c("2023-01-01", "2023-01-02", "2023-01-03", "2023-01-04", "2023-01-05"),
  CTR = c(2.3, 2.7, 2.0, 2.4, 2.6)
)
website_data <- website_data[order(-website_data$CTR), ]
ggplot(website_data,
  aes(x = reorder(Date, CTR), y = CTR, fill = Date)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Top Days by Click-through Rate",
    x = "Date",
    y = "Click-through Rate (%)"
  ) +
  theme_minimal()
```

Output:



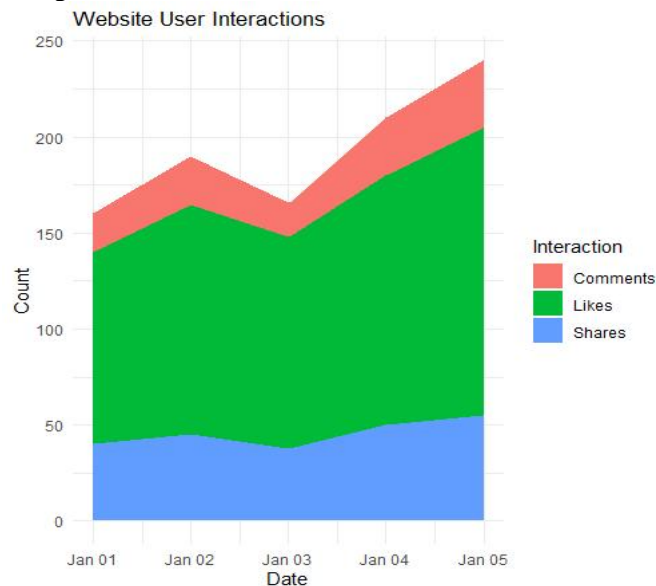
5.C:Develop a stacked area chart to display the distribution of user interactions (likes, shares, comments) on a website.

Code:

```
library(ggplot2)
library(tidyr)
interaction_data <- data.frame(
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03", "2023-01-04", "2023-01-05")),
  Likes = c(100, 120, 110, 130, 150),
  Shares = c(40, 45, 38, 50, 55),
  Comments = c(20, 25, 18, 30, 35)
)
interaction_long <- pivot_longer(
  interaction_data,
  cols = c(Likes, Shares, Comments),
  names_to = "Interaction",
  values_to = "Count"
)
```

```
ggplot(interaction_long,
       aes(x = Date, y = Count, fill = Interaction)) +
  geom_area() +
  labs(
    title = "Website User Interactions",
    x = "Date",
    y = "Count"
  ) +
  theme_minimal()
```

Output:



5.D: Build a Tableau dashboard with the line chart and bar chart for interactive exploration of website traffic data.

Code:

```
library(shiny)
library(plotly)
website_data <- data.frame(
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03", "2023-01-04", "2023-01-05")),
  PageViews = c(1500, 1600, 1400, 1650, 1800),
  CTR = c(2.3, 2.7, 2.0, 2.4, 2.6)
)
ui <- fluidPage(
  titlePanel("Website Traffic Dashboard"),
  fluidRow(
    column(6,
      plotlyOutput("linePlot")
    ),
    column(6,
      plotlyOutput("barPlot")
    )
  )
)
server <- function(input, output){
  output$linePlot <- renderPlotly({
    plot_ly(
      website_data,
```

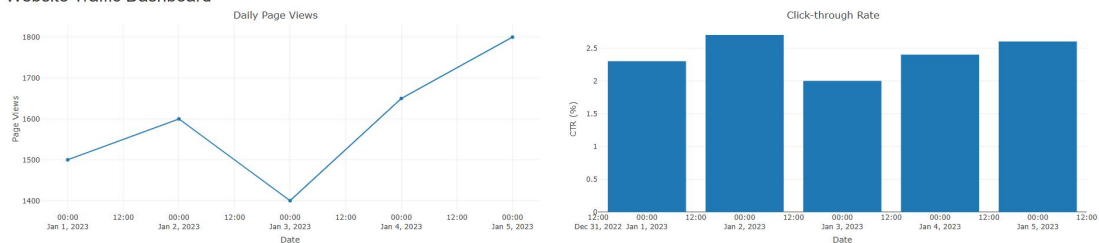
```

x = ~Date,
y = ~PageViews,
type = "scatter",
mode = "lines+markers"
) %>%
layout(
  title = "Daily Page Views",
  xaxis = list(title = "Date"),
  yaxis = list(title = "Page Views")
)
})
output$barPlot <- renderPlotly({
  plot_ly(
    website_data,
    x = ~Date,
    y = ~CTR,
    type = "bar"
  ) %>%
  layout(
    title = "Click-through Rate",
    xaxis = list(title = "Date"),
    yaxis = list(title = "CTR (%)")
  )
})
}
shinyApp(ui = ui, server = server)

```

Output:

Website Traffic Dashboard



Set 6

Product Sales Analysis

Dataset: Monthly Product Sales

Product ID	Product Name	January Sales	February Sales	March Sales
1	Product A	2000	2200	2400
2	Product B	1500	1800	1600
3	Product C	1200	1400	1100

6.A: Create a grouped bar chart to visualize the sales of each product for the first quarter. Label the chart elements.

Code:

```

library(ggplot2)
library(tidyr)
sales_data <- data.frame(
  ProductID = c(1,2,3),

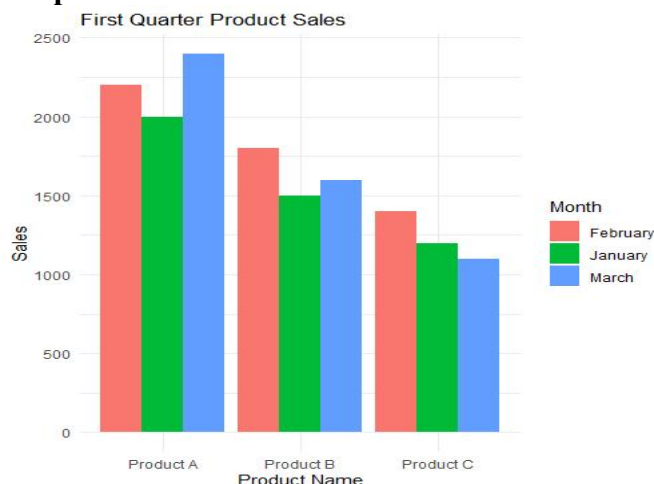
```

```

    ProductName = c("Product A","Product B","Product C"),
    January = c(2000,1500,1200),
    February = c(2200,1800,1400),
    March = c(2400,1600,1100)
  )
sales_long <- pivot_longer(
  sales_data,
  cols = c(January, February, March),
  names_to = "Month",
  values_to = "Sales"
)
ggplot(sales_long,
  aes(x = ProductName, y = Sales, fill = Month)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(
    title = "First Quarter Product Sales",
    x = "Product Name",
    y = "Sales"
  ) +
  theme_minimal()

```

Output:



6.B:Generate a stacked area chart to represent the overall sales trend for all products over the three months.

Code:

```

library(ggplot2)
library(tidyr)
sales_data <- data.frame(
  ProductName = c("Product A","Product B","Product C"),
  January = c(2000,1500,1200),
  February = c(2200,1800,1400),
  March = c(2400,1600,1100)
)
sales_long <- pivot_longer(
  sales_data,
  cols = c(January, February, March),
  names_to = "Month",
  values_to = "Sales"
)

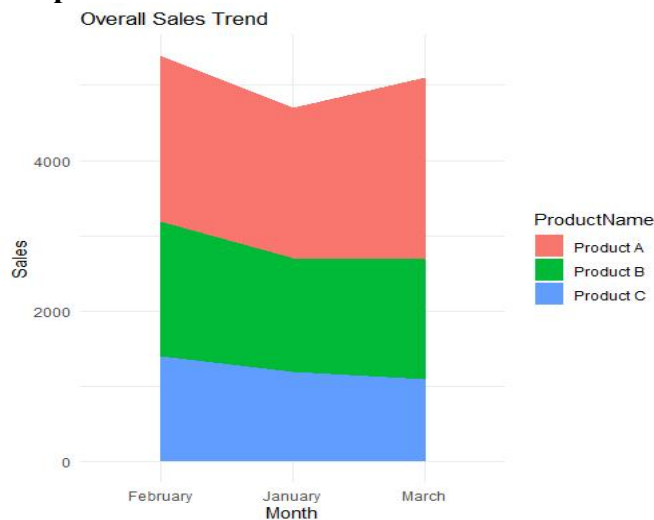
```

```

)
ggplot(sales_long,
  aes(x = Month, y = Sales, fill = ProductName, group = ProductName)) +
  geom_area() +
  labs(
    title = "Overall Sales Trend",
    x = "Month",
    y = "Sales"
  ) +
  theme_minimal()

```

Output:



6.C:Build a table to show the monthly sales data for each product. Label the table elements.

Code:

```

sales_data <- data.frame(
  ProductID = c(1,2,3),
  ProductName = c("Product A","Product B","Product C"),
  January = c(2000,1500,1200),
  February = c(2200,1800,1400),
  March = c(2400,1600,1100)
)
print(sales_data)

```

Output:

ProductID	ProductName	January	February	March
1	Product A	2000	2200	2400
2	Product B	1500	1800	1600
3	Product C	1200	1400	1100

6.D:Develop a Tableau dashboard combining the grouped bar chart, stacked area chart, and the table for interactive exploration of sales data.

Code:

```

library(shiny)
library(plotly)
library(DT)
library(tidyr)

```

```

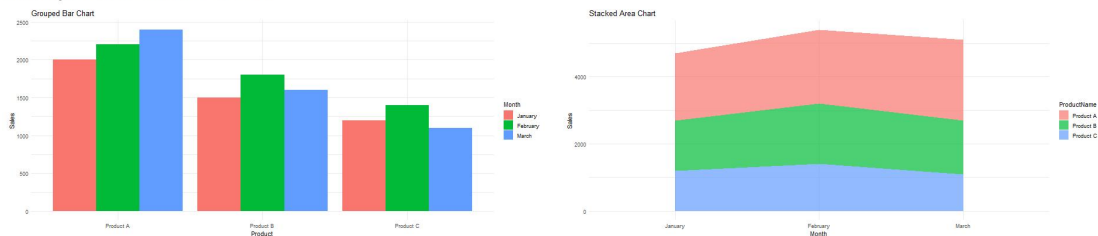
sales_data <- data.frame(
  ProductID = c(1,2,3),
  ProductName = c("Product A","Product B","Product C"),
  January = c(2000,1500,1200),
  February = c(2200,1800,1400),
  March = c(2400,1600,1100)
)
sales_long <- pivot_longer(
  sales_data,
  cols = c(January, February, March),
  names_to = "Month",
  values_to = "Sales"
)
ui <- fluidPage(
  titlePanel("Monthly Product Sales Dashboard"),
  fluidRow(
    column(6,
      plotlyOutput("barPlot")
    ),
    column(6,
      plotlyOutput("areaPlot")
    )
  ),
  br(),
  fluidRow(
    column(12,
      DTOutput("salesTable")
    )
  )
)
server <- function(input, output){
  output$barPlot <- renderPlotly({
    p <- ggplot(sales_long,
      aes(x = ProductName, y = Sales, fill = Month)) +
      geom_bar(stat = "identity", position = "dodge") +
      labs(
        title = "First Quarter Product Sales",
        x = "Product",
        y = "Sales"
      ) +
      theme_minimal()
    ggplotly(p)
  })
  output$areaPlot <- renderPlotly({
    p <- ggplot(sales_long,
      aes(x = Month, y = Sales,
        fill = ProductName,
        group = ProductName)) +
      geom_area() +

```

```
labs(
  title = "Overall Sales Trend",
  x = "Month",
  y = "Sales"
) +
theme_minimal()
ggplotly(p)
})
output$salesTable <- renderDT({
  datatable(sales_data)
})
}
shinyApp(ui = ui, server = server)
```

Output:

Monthly Product Sales Dashboard



Monthly Sales Table

ProductID	ProductName	January	February	March
1.00	Product A	2000.00	2200.00	2400.00
2.00	Product B	1500.00	1800.00	1600.00
3.00	Product C	1200.00	1400.00	1100.00